Computer Architecture
Fall 2026
Project 1: Instruction Level Simulator
Cheikh Lame, Mira Allis, Randy Bauer, Ahmad Rao, Jeremiah Robinson

Machine overview

- **Word size:** 32 bits.
- **Registers:** 32 total, in two files of 16. Each 4-bit register field names one of 16 registers; the opcode (and sub-opcode) decides which file.
  - **Integer registers R0–R15.** R0 always reads as 0; writes to R0 are ignored. R15 is the link register: JAL saves the return address there.
  - **Floating-point registers F0–F15.** Each holds a 32-bit IEEE-754 single-precision value.
- **Memory:** instruction memory and data memory are separate.
  - **Instruction memory** holds one instruction per address. The PC counts instructions, not bytes. The first instruction in the assembly file is at address 0. Addresses are 28 bits.
  - **Data memory** is 2^28 bytes and byte addressable. A word occupies 4 bytes, stored big-endian (most significant byte at the lowest address). Word loads and stores must use an address that is a multiple of 4. An address outside 0 to 2^28 − 1 is an error; the simulator stops and reports it.
- **PC update:** every instruction performs PC ← PC + 1 unless its RTL assigns PC (taken branches, J, JAL, JR). HALT stops execution.
- **Arithmetic:** integers are two's complement. ADD, SUB, and ADDI wrap on overflow; nothing traps.
- **Immediates:** load/store offsets, branch offsets, and the ADDI immediate are sign-extended. The ANDI, ORI, and XORI immediates are zero-extended.
- **Unused fields** must be all zeros. So must the sub-opcode field of any instruction that does not use it.

32-bit instruction formats

R-format — register operations

opcode	rs	rt	rd	sub-opcode	unused
31–28	27–24	23–20	19–16	15–12	11–0
4 bits	4 bits	4 bits	4 bits	4 bits	12 bits

I-format — immediate, load/store, branch

opcode	rs	rt	sub-opcode	immediate / offset
31–28	27–24	23–20	19–16	15–0
4 bits	4 bits	4 bits	4 bits	16 bits

J-format — jumps

opcode	address
31–28	27–0
4 bits	28 bits

HALT format

opcode	unused
31–28	27–0
4 bits	28 bits

Opcodes

Opcode	Instruction class	Format	Description
0000	ALU-R	R	Register-to-register integer ALU operations (sub-opcode selects the operation)
0001	ALU-I	I	Register-immediate integer ALU operations (sub-opcode selects the operation)
0010	LW	I	Load 32-bit integer
0011	SW	I	Store 32-bit integer
0100	BRANCH	I	Conditional branches (sub-opcode selects the comparison)
0101	J	J	Unconditional jump
0110	JAL	J	Jump and save return address
0111	JR	R	Jump to the address held in a register
1000	FLW	I	Load 32-bit floating-point value
1001	FSW	I	Store 32-bit floating-point value
1010	FP-ALU	R	Floating-point arithmetic, compare, and move (sub-opcode selects the operation)
1011	RESERVED	—	Available for future expansion
1100	LUI	I	Load upper immediate
1101	RESERVED	—	Available for future expansion
1110	RESERVED	—	Available for future expansion
1111	HALT	HALT	Stop execution

ALU-R operations (opcode 0000, R-format)

Sub-opcode	Instruction	RTL	Description	Note
0000	ADD rd, rs, rt	R[rd] ← R[rs] + R[rt] PC ← PC + 1	Add	—
0001	SUB rd, rs, rt	R[rd] ← R[rs] - R[rt] PC ← PC + 1	Subtract	—
0010	AND rd, rs, rt	R[rd] ← R[rs] AND R[rt] PC ← PC + 1	Bitwise AND	Used to mask: keeps only the bits that are 1 in both.

Sub-opcode	Instruction	RTL	Description	Note
0011	OR rd, rs, rt	R[rd] ← R[rs] OR R[rt] PC ← PC + 1	Bitwise OR	Used to set selected bits to 1.
0100	XOR rd, rs, rt	R[rd] ← R[rs] XOR R[rt] PC ← PC + 1	Bitwise exclusive OR	Flips selected bits. XOR rd, rs, rs sets rd to 0.
0101	SLT rd, rs, rt	R[rd] ← 1 if R[rs] < R[rt] (signed) else 0 PC ← PC + 1	Set on less than, signed	Does not branch. Stores a 1 or 0 that a later BEQ/BNE can test against R0.
0110	SLTU rd, rs, rt	R[rd] ← 1 if unsigned(R[rs]) < unsigned(R[rt]) else 0 PC ← PC + 1	Set on less than, unsigned	For values that are never negative, such as addresses and counts.
0111	SLL rd, rs, rt	R[rd] ← R[rs] << R[rt][4:0] PC ← PC + 1	Shift left logical	Multiplies by a power of two. The shift amount comes from the low 5 bits of R[rt] (0–31).
1000	SRL rd, rs, rt	R[rd] ← R[rs] >> R[rt][4:0], zeros shifted in PC ← PC + 1	Shift right logical	Divides an unsigned value by a power of two.
1001	SRA rd, rs, rt	R[rd] ← R[rs] >> R[rt][4:0], sign bit shifted in PC ← PC + 1	Shift right arithmetic	Divides a signed value by a power of two; negative numbers stay negative.

Sub-opcodes 1010–1111 are reserved for additional instructions (TBD)

ALU-I operations (opcode 0001, I-format)

Sub-opcode	Instruction	RTL	Description	Note
0000	ADDI rt, rs, imm	R[rt] ← R[rs] + SignExt(imm) PC ← PC + 1	Add immediate	Useful for loop counters and stepping pointers. With rs = R0 it loads a constant from -32,768 to 32,767. ADDI rt, rs, 0 copies a register.
0001	ANDI rt, rs, imm	R[rt] ← R[rs] AND ZeroExt(imm) PC ← PC + 1	AND with immediate	Immediate is zero-extended, since a mask is never negative.
0010	ORI rt, rs, imm	R[rt] ← R[rs] OR ZeroExt(imm) PC ← PC + 1	OR with immediate	Pairs with LUI to build a full 32-bit constant.
0011	XORI rt, rs, imm	R[rt] ← R[rs] XOR ZeroExt(imm) PC ← PC + 1	XOR with immediate	Flips selected bits.

Sub-opcodes 0100–1111 are reserved for additional instructions (TBD)

BRANCH operations (opcode 0100, I-format)

All branches use the form Bxx rs, rt, offset. The offset counts instructions, measured from the instruction after the branch.

Sub-opcode	Instruction	RTL	Description	Note
0000	BEQ rs, rt, offset	if R[rs] = R[rt] then PC ← PC + 1 + SignExt(offset) else PC ← PC + 1	Branch if equal	BEQ rs, R0 branches if rs is zero.

Sub-opcode	Instruction	RTL	Description	Note
0001	BNE rs, rt, offset	if R[rs] ≠ R[rt] then PC ← PC + 1 + SignExt(offset) else PC ← PC + 1	Branch if not equal	—
0010	BLT rs, rt, offset	if R[rs] < R[rt] (signed) then PC ← PC + 1 + SignExt(offset) else PC ← PC + 1	Branch if less than, signed	For "greater than," swap the operands: a > b is the same as b < a.
0011	BGE rs, rt, offset	if R[rs] ≥ R[rt] (signed) then PC ← PC + 1 + SignExt(offset) else PC ← PC + 1	Branch if greater than or equal, signed	For "less than or equal," swap the operands.
0100	BLTU rs, rt, offset	if unsigned(R[rs]) < unsigned(R[rt]) then PC ← PC + 1 + SignExt(offset) else PC ← PC + 1	Branch if less than, unsigned	For comparing addresses and counts.
0101	BGEU rs, rt, offset	if unsigned(R[rs]) ≥ unsigned(R[rt]) then PC ← PC + 1 + SignExt(offset) else PC ← PC + 1	Branch if greater than or equal, unsigned	For comparing addresses and counts.

Sub-opcodes 0110–1111 are reserved for additional instructions (TBD)

Single-opcode instructions

Opcode	Instruction	Format	RTL	Description	Note
0010	LW rt, offset(rs)	I	R[rt] ← M[R[rs] + SignExt(offset)] PC ← PC + 1	Load a 32-bit integer from data memory	The address must be a multiple of 4. Sub-opcode field must be 0000.
0011	SW rt, offset(rs)	I	M[R[rs] + SignExt(offset)] ← R[rt] PC ← PC + 1	Store a 32-bit integer to data memory	The address must be a multiple of 4. Sub-opcode field must be 0000.
0101	J address	J	PC ← address	Jump to an absolute instruction address	28 address bits can reach every instruction.
0110	JAL address	J	R[15] ← PC + 1 PC ← address	Jump to a fixed address and save the return address in R15	Starts a function call. The return address always goes to R15. A function that calls another function must save R15 to memory first, or it loses its own return address.
0111	JR rs	R	PC ← R[rs]	Jump to the address held in a register	JR R15 returns from a function. The rt, rd, and sub-opcode fields must be 0.
1000	FLW ft, offset(rs)	I	F[rt] ← M[R[rs] + SignExt(offset)] PC ← PC + 1	Load a 32-bit floating-point value from data memory	The base address comes from an integer register; the destination is a floating-point register. The 32 bits are copied unchanged.

Opcode	Instruction	Format	RTL	Description	Note
1001	FSW ft, offset(rs)	I	M[R[rs] + SignExt(offset)] ← F[rt] PC ← PC + 1	Store a 32-bit floating-point value to data memory	The base address comes from an integer register; the source is a floating-point register.
1100	LUI rt, imm	I	R[rt] ← imm << 16 (lower 16 bits become 0) PC ← PC + 1	Load upper immediate	Loads the upper half of a 32-bit constant. Follow with ORI rt, rt, lower16 to fill in the lower half. The rs and sub-opcode fields must be 0.
1111	HALT	HALT	Stop execution	End the program	Every benchmark must end with HALT. The simulator then prints instruction counts and writes the output files.

FP-ALU operations (opcode 1010, R-format)

The rs, rt, and rd fields name floating-point registers, except that FLT and FEQ write their result to an integer register R[rd].

Sub-opcode	Instruction	RTL	Description	Note
0000	FADD fd, fs, ft	F[rd] ← F[rs] + F[rt] PC ← PC + 1	Floating-point addition	IEEE-754 single-precision arithmetic.
0001	FSUB fd, fs, ft	F[rd] ← F[rs] - F[rt] PC ← PC + 1	Floating-point subtraction	—
0010	FMUL fd, fs, ft	F[rd] ← F[rs] × F[rt] PC ← PC + 1	Floating-point multiplication	—
0011	FDIV fd, fs, ft	F[rd] ← F[rs] ÷ F[rt] PC ← PC + 1	Floating-point division	Dividing by zero gives infinity or NaN as IEEE-754 defines; it does not stop the program.
0100	FLT rd, fs, ft	R[rd] ← 1 if F[rs] < F[rt] else 0 PC ← PC + 1	Floating-point less-than compare	The result goes to an integer register so a branch can test it: FLT R1, F2, F3 then BNE R1, R0, target. Needed by the floating-point MAXFINDER. Any comparison with NaN gives 0.
0101	FEQ rd, fs, ft	R[rd] ← 1 if F[rs] = F[rt] else 0 PC ← PC + 1	Floating-point equality compare	Result goes to an integer register, like FLT.
0110	FMOV fd, fs	F[rd] ← F[rs] PC ← PC + 1	Copy a floating-point register	The rt field must be 0. There is no floating-point zero register, so this is the way to copy one FP register into another.

Sub-opcodes 0111–1111 are reserved for additional instructions (TBD)